`<delay_value>` Specifies the number of cycles to delay after the instruction completes. The delay_value is specified as a value (see Section 11.3.2), and in general is between 0 and 31 inclusive (a 5-bit value), however the number of bits is reduced when sideset is enabled via the `.side_set` (see `pioasm_side_set`) directive. If the `<delay_value>` is not present, then the instruction has no delay.

NOTE

pioasm instruction names, keywords and directives are case insensitive; lower case is used in the *Assembly Syntax* sections below, as this is the style used in the SDK.

NOTE

Commas appear in some *Assembly Syntax* sections below, but are entirely optional, e.g. `out pins, 3` may be written `out pins 3`, and `jmp x-- label` may be written as `jmp x--, label`. The *Assembly Syntax* sections below uses the first style in each case as this is the style used in the SDK.

11.3.7. Pseudo-instructions

pioasm provides aliases for certain instructions, as a convenience:

`nop` Assembles to `mov y, y`. No side effect, but a useful vehicle for a side-set operation or an extra delay.

11.4. Instruction Set

11.4.1. Summary

PIO instructions are 16 bits long, and use the following encoding:

Table 980. PIO instruction encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP	0	0	0	Delay/side-set					Condition			Address				
WAIT	0	0	1	Delay/side-set					Pol	Source		Index				
IN	0	1	0	Delay/side-set					Source			Bit count				
OUT	0	1	1	Delay/side-set					Destination			Bit count				
PUSH	1	0	0	Delay/side-set					0	IfF	Blk	0	0	0	0	0
MOV	1	0	0	Delay/side-set					0	0	0	1	IdxI	0	Index	
PULL	1	0	0	Delay/side-set					1	IfE	Blk	0	0	0	0	0
MOV	1	0	0	Delay/side-set					1	0	0	1	IdxI	0	Index	
MOV	1	0	1	Delay/side-set					Destination			Op		Source		
IRQ	1	1	0	Delay/side-set					0	Clr	Wait	IdxMode		Index		
SET	1	1	1	Delay/side-set					Destination			Data				

All PIO instructions execute in one clock cycle.

The function of the 5-bit `Delay/side-set` field depends on the state machine's `SIDESET_COUNT` configuration:

- Up to 5 LSBs (5 minus `SIDESET_COUNT`) encode a number of idle cycles inserted between this instruction and the next.

- Up to 5 MSBs, set by `SIDESET_COUNT`, encode a side-set (Section 11.5.1), which can assert a constant onto some GPIOs, concurrently with main instruction execution.

11.4.2. JMP

11.4.2.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`JMP`	0	0	0	Delay/side-set					Condition			Address				

11.4.2.2. Operation

Set program counter to `Address` if `Condition` is true, otherwise no operation.

Delay cycles on a `JMP` always take effect, whether `Condition` is true or false, and they take place *after* `Condition` is evaluated and the program counter is updated.

- Condition:
  - 000: *(no condition)*: Always
  - 001: `!X`: scratch X zero
  - 010: `X--`: scratch X non-zero, prior to decrement
  - 011: `!Y`: scratch Y zero
  - 100: `Y--`: scratch Y non-zero, prior to decrement
  - 101: `X!=Y`: scratch X not equal scratch Y
  - 110: `PIN`: branch on input pin
  - 111: `!OSRE`: output shift register not empty
- Address: Instruction address to jump to. In the instruction encoding this is an absolute address within the PIO instruction memory

`JMP PIN` branches on the GPIO selected by `EXECCTRL_JMP_PIN`, a configuration field which selects one out of the maximum of 32 GPIO inputs visible to a state machine, independently of the state machine's other input mapping. The branch is taken if the GPIO is high.

`!OSRE` compares the bits shifted out since the last `PULL` with the shift count threshold configured by `SHIFTCTRL_PULL_THRESH`. This is the same threshold used by autopull (Section 11.5.4).

`JMP X--` and `JMP Y--` always decrement scratch register X or Y, respectively. The decrement is not conditional on the current value of the scratch register. The branch is conditioned on the *initial* value of the register, i.e. before the decrement took place: if the register is initially nonzero, the branch is taken.

11.4.2.3. Assembler syntax

```
jmp (<cond>) <target>
```

where:

`<cond>` An optional condition listed above (e.g. `!x` for scratch X zero). If a condition code is not specified, the branch is always taken.

`<target>` A program label or value (see Section 11.3.2) representing instruction offset within the program (the first instruction being offset 0). Because the PIO JMP instruction uses absolute addresses in the PIO instruction memory, JMPs need to be adjusted based on the program load offset at runtime. This is handled for you when loading a program with the SDK, but care should be taken when encoding JMP instructions for use by `OUT EXEC`.

11.4.3. WAIT

11.4.3.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`WAIT`	0	0	1	Delay/side-set					Pol	Source		Index				

11.4.3.2. Operation

Stall until some condition is met.

Like all stalling instructions (Section 11.2.5), delay cycles begin after the instruction *completes*. That is, if any delay cycles are present, they do not begin counting until *after* the wait condition is met.

- Polarity:
  - 1: wait for a 1.
  - 0: wait for a 0.
- Source: what to wait on. Values are:
  - 00: `GPIO`: System GPIO input selected by `Index`. This is an absolute GPIO index, and is not affected by the state machine's input IO mapping.
  - 01: `PIN`: Input pin selected by `Index`. This state machine's input IO mapping is applied first, and then `Index` selects which of the mapped bits to wait on. In other words, the pin is selected by adding `Index` to the `PINCTRL_IN_BASE` configuration, modulo 32.
  - 10: `IRQ`: PIO IRQ flag selected by `Index`
  - 11: `JMPPIN`: wait on the pin indexed by the `PINCTRL_JMP_PIN` configuration, plus an Index in the range 0-3, all modulo 32. Other values of Index are reserved.
- Index: which pin or bit to check.

`WAIT x IRQ` behaves slightly differently from other `WAIT` sources:

- If `Polarity` is 1, the selected IRQ flag is cleared by the state machine upon the wait condition being met.
- The flag index is decoded in the same way as the `IRQ` index field, decoding down from the two MSBs (aligning with the `IRQ` instruction IdxMode field):
  - `00`: the three LSBs are used directly to index the IRQ flags in this PIO block.
  - `01` (`PREV`), the instruction references an IRQ from the next-lower-numbered PIO in the system, wrapping to the highest-numbered PIO if this is PIO0.
  - `10` (`REL`), the state machine ID (0…3) is added to the IRQ index, by way of modulo-4 addition on the two LSBs. For example, state machine 2 with a flag value of `0x11` will wait on flag 3, and a flag value of `0x13` will wait on flag 1. This allows multiple state machines running the same program to synchronise with each other.

- 11 (NEXT), the instruction references an IRQ from the next-higher-numbered PIO in the system, wrapping to PIO0 if this is the highest-numbered PIO.

CAUTION

WAIT 1 IRQ x should not be used with IRQ flags presented to the interrupt controller, to avoid a race condition with a system interrupt handler

11.4.3.3. Assembler syntax

```
wait <polarity> gpio <gpio_num>
```

```
wait <polarity> pin <pin_num>
```

```
wait <polarity> irq (prev | next) <irq_num> (rel)
```

```
wait <polarity> jmppin (+ <pin_offset>)
```

where:

<polarity>	A value (see Section 11.3.2) specifying the polarity (either 0 or 1).
<pin_num>	A value (see Section 11.3.2) specifying the input pin number (as mapped by the SM input pin mapping).
<gpio_num>	A value (see Section 11.3.2) specifying the actual GPIO pin number.
<irq_num> (rel)	A value (see Section 11.3.2) specifying The IRQ number to wait on (0-7). If rel is present, then the actual IRQ number used is calculating by replacing the low two bits of the IRQ number (irq_num_{10}) with the low two bits of the sum (irq_num_{10} + sm_num_{10}) where sm_num_{10} is the state machine number.
prev	To wait on the IRQ on the next lower numbered PIO block instead of on the current PIO block
next	To wait on the IRQ on the next higher numbered PIO block instead of on the current PIO block
<pin_offset>	A value (see Section 11.3.2) added to the jmp_pin to get the actual pin number.

11.4.4. IN

11.4.4.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IN	0	1	0	Delay/side-set					Source			Bit count				

11.4.4.2. Operation

Shift `Bit count` bits from `Source` into the Input Shift Register (ISR). Shift direction is configured for each state machine by `SHIFTCTRL_IN_SHIFTDIR`. Additionally, increase the input shift count by `Bit count`, saturating at 32.

- Source:
  - 000: `PINS`
  - 001: `X` (scratch register X)
  - 010: `Y` (scratch register Y)
  - 011: `NULL` (all zeroes)
  - 100: Reserved
  - 101: Reserved
  - 110: `ISR`
  - 111: `OSR`
- Bit count: How many bits to shift into the ISR. 1…32 bits, 32 is encoded as `00000`

If automatic push is enabled, `IN` will also push the ISR contents to the RX FIFO if the push threshold is reached (`SHIFTCTRL_PUSH_THRESH`). `IN` still executes in one cycle, whether an automatic push takes place or not. The state machine will stall if the RX FIFO is full when an automatic push occurs. An automatic push clears the ISR contents to all-zeroes, and clears the input shift count. See Section 11.5.4.

`IN` always uses the least significant `Bit count` bits of the source data. For example, if `PINCTRL_IN_BASE` is set to 5, the instruction `IN PINS, 3` will take the values of pins 5, 6 and 7, and shift these into the ISR. First the ISR is shifted to the left or right to make room for the new input data, then the input data is copied into the gap this leaves. The bit order of the input data is not dependent on the shift direction.

`NULL` can be used for shifting the ISR's contents. For example, UARTs receive the LSB first, so must shift to the right. After 8 `IN PINS, 1` instructions, the input serial data will occupy bits 31…24 of the ISR. An `IN NULL, 24` instruction will shift in 24 zero bits, aligning the input data at ISR bits 7…0. Alternatively, the processor or DMA could perform a byte read from FIFO address + 3, which would take bits 31…24 of the FIFO contents.

11.4.4.3. Assembler syntax

```
in <source>, <bit_count>
```

where:

`<source>`	One of the sources specified above.
`<bit_count>`	A value (see Section 11.3.2) specifying the number of bits to shift (valid range 1-32).

11.4.5. OUT

11.4.5.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`OUT`	0	1	1	Delay/side-set					Destination			Bit count				

11.4.5.2. Operation

Shift `Bit count` bits out of the Output Shift Register (OSR), and write those bits to `Destination`. Additionally, increase the output shift count by `Bit count`, saturating at 32.

- Destination:
  - 000: `PINS`
  - 001: `X` (scratch register X)
  - 010: `Y` (scratch register Y)
  - 011: `NULL` (discard data)
  - 100: `PINDIRS`
  - 101: `PC`
  - 110: `ISR` (also sets ISR shift counter to `Bit count`)
  - 111: `EXEC` (Execute OSR shift data as instruction)
- Bit count: how many bits to shift out of the OSR. 1…32 bits, 32 is encoded as `00000`

A 32-bit value is written to `Destination`: the lower `Bit count` bits come from the OSR, and the remainder are zeroes. This value is the least significant `Bit count` bits of the OSR if `SHIFTCTRL_OUT_SHIFTDIR` is to the right, otherwise it is the most significant bits.

`PINS` and `PINDIRS` use the `OUT` pin mapping, as described in Section 11.5.6.

If automatic pull is enabled, the OSR is automatically refilled from the TX FIFO if the pull threshold, `SHIFTCTRL_PULL_THRESH`, is reached. The output shift count is simultaneously cleared to 0. In this case, the `OUT` will stall if the TX FIFO is empty, but otherwise still executes in one cycle. The specifics are given in Section 11.5.4.

`OUT EXEC` allows instructions to be included inline in the FIFO datastream. The `OUT` itself executes on one cycle, and the instruction from the OSR is executed on the next cycle. There are no restrictions on the types of instructions which can be executed by this mechanism. Delay cycles on the initial `OUT` are ignored, but the executee may insert delay cycles as normal.

`OUT PC` behaves as an unconditional jump to an address shifted out from the OSR.

11.4.5.3. Assembler syntax

```
out <destination>, <bit_count>
```

where:

`<destination>`	One of the destinations specified above.
`<bit_count>`	A value (see Section 11.3.2) specifying the number of bits to shift (valid range 1-32).

11.4.6. PUSH

11.4.6.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`PUSH`	1	0	0	Delay/side-set					0	IfF	Blk	0	0	0	0	0

11.4.6.2. Operation

Push the contents of the ISR into the RX FIFO, as a single 32-bit word. Clear ISR to all-zeroes.

- `IfFull`: If 1, do nothing unless the total input shift count has reached its threshold, `SHIFTCTRL_PUSH_THRESH` (the same as for autopush; see Section 11.5.4).
- `Block`: If 1, stall execution if RX FIFO is full.

`PUSH IFFULL` helps to make programs more compact, like autopush. It is useful in cases where the `IN` would stall at an inappropriate time if autopush were enabled, e.g. if the state machine is asserting some external control signal at this point.

The PIO assembler sets the `Block` bit by default. If the `Block` bit is not set, the `PUSH` does not stall on a full RX FIFO, instead continuing immediately to the next instruction. The FIFO state and contents are unchanged when this happens. The ISR is still cleared to all-zeroes, and the `FDEBUG_RXSTALL` flag is set (the same as a blocking `PUSH` or autopush to a full RX FIFO) to indicate data was lost.

NOTE

The operation of the `PUSH` instruction is undefined when SM0_SHIFTCTRL.FJOIN_RX_PUT or FJOIN_RX_GET is set — see Section 11.4.8 and Section 11.4.9 for details of the `PUT` and `GET` instruction which can be used in this state.

11.4.6.3. Assembler syntax

```
push (iffull)
```

```
push (iffull) block
```

```
push (iffull) noblock
```

where:

`iffull`	Equivalent to `IfFull == 1` above. i.e. the default if this is not specified is `IfFull == 0`.
`block`	Equivalent to `Block == 1` above. This is the default if neither `block` nor `noblock` is specified.
`noblock`	Equivalent to `Block == 0` above.

11.4.7. PULL

11.4.7.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`PULL`	1	0	0	Delay/side-set					1	IfE	Blk	0	0	0	0	0

11.4.7.2. Operation

Load a 32-bit word from the TX FIFO into the OSR.

- `IfEmpty`: If 1, do nothing unless the total output shift count has reached its threshold, `SHIFTCTRL_PULL_THRESH` (the same as for autopull; see Section 11.5.4).
- `Block`: If 1, stall if TX FIFO is empty. If 0, pulling from an empty FIFO copies scratch X to OSR.

Some peripherals (UART, SPI, etc.) should halt when no data is available, and pick it up as it comes in; others (I2S) should clock continuously, and it is better to output placeholder or repeated data than to stop clocking. This can be achieved with the `Block` parameter.

A non-blocking `PULL` on an empty FIFO has the same effect as `MOV OSR, X`. The program can either preload scratch register X with a suitable default, or execute a `MOV X, OSR` after each `PULL NOBLOCK`, so that the last valid FIFO word will be recycled until new data is available.

`PULL IFEMPTY` is useful if an `OUT` with autopull would stall in an inappropriate location when the TX FIFO is empty. `IfEmpty` permits some of the same program simplifications as autopull: for example, the elimination of an outer loop counter. However, the stall occurs at a controlled point in the program.

NOTE

When autopull is enabled, any `PULL` instruction is a no-op when the OSR is full, so that the `PULL` instruction behaves as a barrier. `OUT NULL, 32` can be used to explicitly discard the OSR contents. See Section 11.5.4.2 for more detail.

11.4.7.3. Assembler syntax

```
pull (ifempty)
```

```
pull (ifempty) block
```

```
pull (ifempty) noblock
```

where:

`ifempty`	Equivalent to `IfEmpty == 1` above. i.e. the default if this is not specified is `IfEmpty == 0`.
`block`	Equivalent to `Block == 1` above. This is the default if neither `block` nor `noblock` is specified.
`noblock`	Equivalent to `Block == 0` above.

11.4.8. MOV (to RX)

11.4.8.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`MOV`	1	0	0	Delay/side-set					0	0	0	1	Idxl	Index		

11.4.8.2. Operation

Write the ISR to a selected RX FIFO entry. The state machine can write the RX FIFO entries in any order, indexed either by the Y register, or an immediate Index in the instruction. Requires the `SHIFTCTRL_FJOIN_RX_PUT` configuration field to be set, otherwise its operation is undefined. The FIFO configuration can be specified for the program via the `.fifo` directive (see `pioasm_fifo`).

If **IdxI** (index by immediate) is set, the RX FIFO's registers are indexed by the two least-significant bits of the Index operand. Otherwise, they are indexed by the two least-significant bits of the Y register. When IdxI is clear, all non-zero values of Index are reserved encodings, and their operation is undefined.

When only `SHIFTCTRL_FJOIN_RX_PUT` is set (in SM0_SHIFTCTRL through SM3_SHIFTCTRL), the system can also read the RX FIFO registers with random access via RXF0_PUTGET0 through RXF0_PUTGET3 (where RXFx indicates which state machine's FIFO is being accessed). In this state, the FIFO register storage is repurposed as status registers, which the state machine can update at any time and the system can read at any time. For example, a quadrature decoder program could maintain the current step count in a status register at all times, rather than pushing to the RX FIFO and potentially blocking.

When both `SHIFTCTRL_FJOIN_RX_PUT` and `SHIFTCTRL_FJOIN_RX_GET` are set, the system can no longer access the RX FIFO storage registers, but the state machine can now put/get the registers in arbitrary order, allowing them to be used as additional scratch storage.

NOTE

> The RX FIFO storage registers have only a single read port and write port, and access through each port is assigned to only one of (system, state machine) at any time.

11.4.8.3. Assembler syntax

```
mov rxfifo[y], isr
```

```
mov rxfifo[<index>], isr
```

where:

`y`	The literal token "y", indicating the RX FIFO entry is indexed by the Y register.
`<index>`	A value (see Section 11.3.2) specifying the RX FIFO entry to write (valid range 0-3).

11.4.9. MOV (from RX)

11.4.9.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`MOV`	1	0	0	Delay/side-set					1	0	0	1	IdxI	Index		

11.4.9.2. Operation

Read the selected RX FIFO entry into the OSR. The PIO state machine can read the FIFO entries in any order, indexed either by the Y register, or an immediate Index in the instruction. Requires the `SHIFTCTRL_FJOIN_RX_GET` configuration field to be set, otherwise its operation is undefined.

If **Idxl** (index by immediate) is set, the RX FIFO's registers are indexed by the two least-significant bits of the Index operand. Otherwise, they are indexed by the two least-significant bits of the Y register. When Idxl is clear, all non-zero values of Index are reserved encodings, and their operation is undefined.

When only `SHIFTCTRL_FJOIN_RX_GET` is set, the system can also write the RX FIFO registers with random access via RXF0_PUTGET0 through RXF0_PUTGET3 (where RXFx indicates which state machine's FIFO is being accessed). In this state, the RX FIFO register storage is repurposed as additional configuration registers, which the system can update at any time and the state machine can read at any time. For example, a UART TX program might use these registers to configure the number of data bits, or the presence of an additional stop bit.

When both `SHIFTCTRL_FJOIN_RX_PUT` and `SHIFTCTRL_FJOIN_RX_GET` are set, the system can no longer access the RX FIFO storage registers, but the state machine can now put/get the registers in arbitrary order, allowing them to be used as additional scratch storage.

NOTE

> The RX FIFO storage registers have only a single read port and write port, and access through each port is assigned to only one of (system, state machine) at any time.

11.4.9.3. Assembler syntax

```
mov osr, rxfifo[y]
```

```
mov osr, rxfifo[<index>]
```

where:

`y`	The literal token "y", indicating the RX FIFO entry is indexed by the Y register.
`<index>`	A value (see Section 11.3.2) specifying the RX FIFO entry to read (valid range 0-3).

11.4.10. MOV

11.4.10.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`MOV`	1	0	1	Delay/side-set					Destination			Op		Source		

11.4.10.2. Operation

Copy data from `Source` to `Destination`.

- Destination:

  - 000: `PINS` (Uses same pin mapping as `OUT`)
  - 001: `X` (Scratch register X)
  - 010: `Y` (Scratch register Y)
  - 011: `PINDIRS` (Uses same pin mapping as `OUT`)
  - 100: `EXEC` (Execute data as instruction)
  - 101: `PC`
  - 110: `ISR` (Input shift counter is reset to 0 by this operation, i.e. empty)
  - 111: `OSR` (Output shift counter is reset to 0 by this operation, i.e. full)
- Operation:
  - 00: None
  - 01: Invert (bitwise complement)
  - 10: Bit-reverse
  - 11: Reserved
- Source:
  - 000: `PINS` (Uses same pin mapping as `IN`)
  - 001: `X`
  - 010: `Y`
  - 011: `NULL`
  - 100: Reserved
  - 101: `STATUS`
  - 110: `ISR`
  - 111: `OSR`

`MOV PC` causes an unconditional jump. `MOV EXEC` has the same behaviour as `OUT EXEC` (Section 11.4.5), and allows register contents to be executed as an instruction. The `MOV` itself executes in 1 cycle, and the instruction in `Source` on the next cycle. Delay cycles on `MOV EXEC` are ignored, but the executee may insert delay cycles as normal.

The `STATUS` source has a value of all-ones or all-zeroes, depending on some state machine status such as FIFO full/empty, configured by `EXECCTRL_STATUS_SEL`.

`MOV` can manipulate the transferred data in limited ways, specified by the `Operation` argument. Invert sets each bit in `Destination` to the logical NOT of the corresponding bit in `Source`, i.e. 1 bits become 0 bits, and vice versa. Bit reverse sets each bit `n` in `Destination` to bit 31 - `n` in `Source`, assuming the bits are numbered 0 to 31.

`MOV dst, PINS` reads pins using the `IN` pin mapping, masked to the number of bits specified by `SHIFTCTRL_IN_COUNT`. The LSB of the read value is the pin indicated by `PINCTRL_IN_BASE`, and each successive bit comes from a higher-numbered pin, wrapping after 31. Result bits greater than the width specified by `SHIFTCTRL_IN_COUNT` configuration are 0.

`MOV PINDIRS, src` is not supported on PIO version 0.

11.4.10.3. Assembler syntax

```
mov <destination>, (op) <source>
```

where:

`<destination>`	One of the destinations specified above.
`op`	If present, is: `!` or `~` for NOT (Note: this is always a bitwise NOT) `::` for bit reverse
`<source>`	One of the sources specified above.

11.4.11. IRQ

11.4.11.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`IRQ`	1	1	0	Delay/side-set					0	Clr	Wait	IdxMode		Index		

11.4.11.2. Operation

Set or clear the IRQ flag selected by `Index` argument.

- Clear: if 1, clear the flag selected by `Index`, instead of raising it. If `Clear` is set, the `Wait` bit has no effect.
- Wait: if 1, halt until the raised flag is lowered again, e.g. if a system interrupt handler has acknowledged the flag.
- Index: specifies an IRQ index from 0-7. This IRQ flag will be set/cleared depending on the Clear bit.
- IdxMode: modify the behaviour if the Index field, either modifying the index, or indexing IRQ flags from a different PIO block:
  - `00`: the three LSBs are used directly to index the IRQ flags in this PIO block.
  - `01` (`PREV`): the instruction references an IRQ flag from the next-lower-numbered PIO in the system, wrapping to the highest-numbered PIO if this is PIO0.
  - `10` (`REL`): the state machine ID (0…3) is added to the IRQ flag index, by way of modulo-4 addition on the two LSBs. For example, state machine 2 with a flag value of '0x11' will wait on flag 3, and a flag value of '0x13' will wait on flag 1. This allows multiple state machines running the same program to synchronise with each other.
  - `11` (`NEXT`): the instruction references an IRQ flag from the next-higher-numbered PIO in the system, wrapping to PIO0 if this is the highest-numbered PIO.

All IRQ flags 0-7 can be routed out to system level interrupts, on either of the PIO's two external interrupt request lines, configured by `IRQ0_INTE` and `IRQ1_INTE`.

The modulo addition mode (`REL`) allows relative addressing of 'IRQ' and 'WAIT' instructions, for synchronising state machines which are running the same program. Bit 2 (the third LSB) is unaffected by this addition.

The `NEXT`/`PREV` modes can be used to synchronise between state machines in different PIO blocks. If these state machines' clocks are divided, their clock dividers must be the same, and must have been synchronised by writing CTRL.NEXTPREV_CLKDIV_RESTART in addition to the relevant NEXT_PIO_MASK/PREV_PIO_MASK bits. Note that the cross-PIO connection is severed between PIOs with different accessibility to Non-secure code, as per ACCESSCTRL.

If `Wait` is set, `Delay` cycles do not begin until after the wait period elapses.

11.4.11.3. Assembler syntax

```
irq (prev | next) <irq_num> (rel)
```

```
irq (prev | next) set <irq_num> (rel)
```

```
irq (prev | next) nowait <irq_num> (rel)
```

```
irq (prev | next) wait <irq_num> (rel)
```

```
irq (prev | next) clear <irq_num> (rel)
```

where:

`<irq_num> (rel)`	A value (see Section 11.3.2) specifying the IRQ number to target (0-7). If `rel` is present, then the actual IRQ number used is calculated by replacing the low two bits of the IRQ number (irq_num_{10}) with the low two bits of the sum (irq_num_{10} + sm_num_{10}) where sm_num_{10} is the state machine number.
`irq`	Set the IRQ without waiting.
`irq set`	Set the IRQ without waiting.
`irq nowait`	Set the IRQ without waiting.
`irq wait`	Set the IRQ and wait for it to be cleared before proceeding.
`irq clear`	Clear the IRQ.
`prev`	To target the IRQ on the next lower numbered PIO block instead of the current PIO block
`next`	To target the IRQ on the next higher numbered PIO block instead of the current PIO block

11.4.12. SET

11.4.12.1. Encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
`SET`	1	1	1	Delay/side-set					Destination			Data				

11.4.12.2. Operation

Write immediate value `Data` to `Destination`.

- Destination: